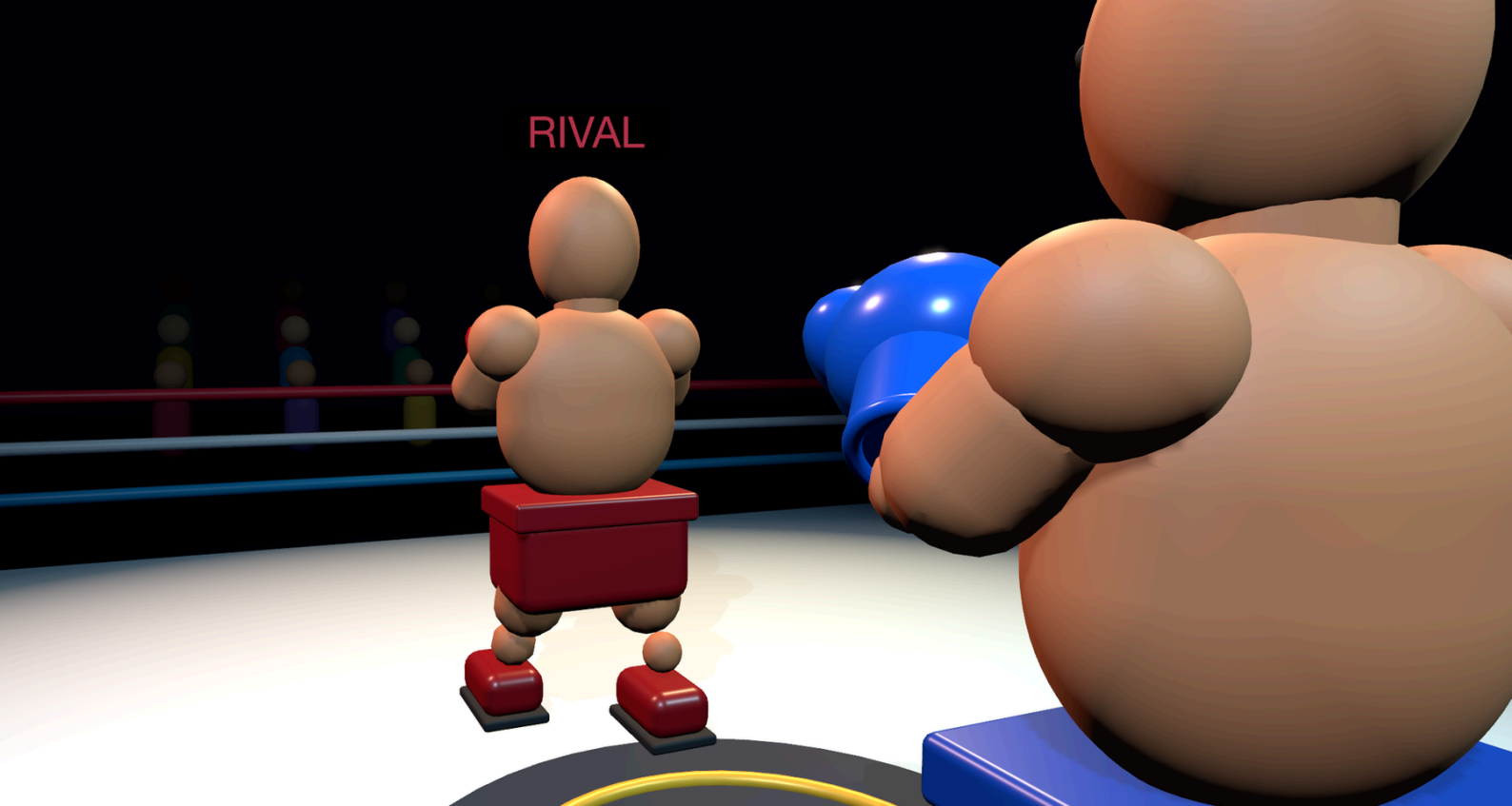




PROJECT PORTFOLIO · iOS GAME

RIVAL

플레이어의 습관을 읽는 3D 복싱 게임

개인 프로젝트 · 기획 / 게임 시스템 / iOS 개발 / 테스트 / App Store 출시

Swift 6 · SwiftUI · SceneKit · GameKit · XCTest

acertainromance401 · 2026

단순히 강해지는 AI가 아니라, 나를 읽는 상대

App Store 정식 출시 · 2026-08-27 · iPhone · iOS 17+

핵심 질문

“같은 패턴을 반복하면 상대가 그 습관을 기억하고 공략하는 복싱 게임을 만들 수 있을까?”

App Store 출시 완료 한국 App Store에 유료 앱으로 정식 출시했습니다. · 출시 버전 1.0 · 게임/스포츠/액션 · ₩4,400

문제

일반적인 난이도 상승은 체력·공격력 증가에 머물러 플레이 어에게 “학습당하고 있다”는 감각을 주기 어렵습니다.

해결

공격, 가드, 더킹, 전후 이동 비율을 기기 안에서 누적하고 라운드와 표본 신뢰도에 따라 RIVAL의 행동 가중치를 조정했습니다.

프로젝트 한눈에 보기

8

서로 다른 공격

3

전투 스타일

10

해금 의상

3

지원 언어

담당 범위

영역	구현 내용
기획	핵심 루프, 공격 체계, 방향 더킹, 난이도 곡선, 보상 구조
게임 로직	프레임 기반 전투 엔진, 적중·가드·카운터·탈진·라운드 판정
클라이언트	SwiftUI 터치 HUD, SceneKit 3D 링·복서·카메라·연출
AI	행동 통계, 적응도, 상황별 가중치, 로컬 영속화와 패배 롤백
품질	XCTest/UI 테스트, 다국어, VoiceOver, 햅틱, 실기기·App Store 배포

결과 C++ TUI에서 시작한 규칙 실험을 웹·Python 프로토타입으로 검증한 뒤, 네이티브 iOS 3D 게임으로 완성해 App Store에 출시했습니다.

두 개의 공격 버튼으로 만드는 8가지 선택

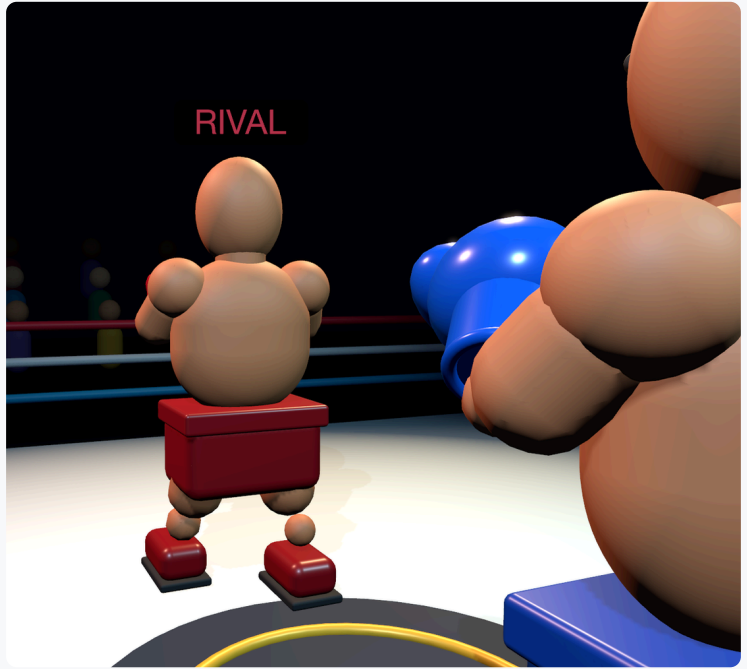
버튼 수는 줄이고, 방향·타이밍·거리의 판단은 늘렸습니다.

조합 입력

홀드	JAB	CROSS
없음	잡	크로스
가드	L 어퍼	R 어퍼
L 더킹	L 바디	R 훅
R 더킹	L 훅	R 바디

설계 의도

모바일 화면을 버튼으로 가득 채우지 않으면서도, 방어 자세와 펀치의 조합을 익히는 손맛을 남겼습니다.



SceneKit으로 생성한 실제 인게임 복서와 링

전투를 읽게 만드는 규칙

방향 더킹 공격 손에 맞는 방향으로 피해야 성공하며, 바디 공격에는 통하지 않습니다.

헛스윙 노출 사거리 밖 공격은 빈틈을 만들고 다음 피격 피해를 키웁니다.

반복 기술 약화 동일 기술을 세 번째부터 반복하면 피해가 단계적으로 감소합니다.

가드 파괴 체력 피해를 줄이는 대신 스테미나를 소모하고, 0이 되면 탈진합니다.

카운터 정확한 회피 뒤 제한 시간 안에 바디·훅을 연결하면 추가 피해가 발생합니다.

승리 기반 진행 플레이어가 이긴 경우에만 라운드와 AI 적을 단계가 올라갑니다.

플레이 스타일

스타일	강점	트레이드오프
인파이터	균형·연타	낮은 공격 스테미나 비용으로 빠른 연계에 유리
아웃복싱	리치·풋워크	빠르고 길지만 공격력이 낮음
슬러거	한 방·가드	강한 공격력과 가드 효율 대신 짧은 리치와 느린 이동

설명 가능한 온디바이스 적응형 AI

외부 API와 플레이 데이터 전송 없이, 현재 기기에서만 동작합니다.

01 · 관찰

8종 공격, 가드, 좌우 더킹, 백스텝, 압박·후퇴를 누적



02 · 해석

행동 비율과 표본 강도, 현재 라운드로 적응도 계산



03 · 대응

거리·상태·습관을 반영한 가중 확률로 다음 행동 선택

적응도 모델

$$\text{adaptation} = \text{clamp}(0.18 + 0.16 \times (\text{round} - 1) + 0.58 \times \text{sampleStrength}, 0.18, 1.0)$$

관찰 → 대응 예시

플레이어 습관	RIVAL의 가중치 변화
가드 유지	바디샷 증가
오른쪽 더킹 반복	오른손 공격 증가
잡 반복	크로스·훅·회피 증가
후퇴·백스텝	잡·크로스 추격 증가
같은 공격 3회+	좌우 더킹 증가

데이터 원칙

Local-first UserDefaults에 Codable JSON으로 저장
Privacy 이름·연락처·위치·광고 식별자 수집 없음
Resilience 저장값이 없거나 손상되면 빈 메모리로 시작
Fairness 패배 라운드의 신규 습관은 시작 시점으로 롤백
Explainability 신경망이 아닌 명시적 규칙과 임계값으로 동작

왜 강화학습 모델을 쓰지 않았나

초기 제품에서는 오프라인 동작, 디버깅 가능성, 작은 앱 크기, 플레이 감각의 직접 조정을 우선했습니다. 런타임 AI와 개발 과정의 생성형 AI 사용도 명확히 분리했습니다.

판정과 표현을 분리한 테스트 가능한 구조

렌더링 없이도 전투 규칙과 AI를 검증할 수 있도록 책임을 나눴습니다.



기술 선택

기술	역할	선택 이유
Swift 6	전투 코어·상태 모델	타입 안정성과 엄격한 동시성 검증
SwiftUI	가로형 HUD·메뉴·접근성	상태 기반 UI와 빠른 반복 개발
SceneKit	3D 링·복서·조명·카메라	외부 모델 없이 코드 기반 지오메트리 생성
GameKit	리더보드	별도 서버 없이 Apple 플랫폼 기능 연동
XCTest	전투·AI·회귀 테스트	렌더러와 분리된 결정 로직 직접 검증
XcodeGen	프로젝트 정의	생성 프로젝트 대신 <code>project.yml</code> 을 단일 원본으로 유지

제품 기능으로 확장된 구조

체육관 고정 샌드백, 타격 반응, 코치 팁, 무제한 콤보 연습 보상 승수·연승·무피격·스타일 숙련·체육관 시간으로 10개 의 상 해금	랭킹 Game Center 총 승수와 최고 라운드 표시 현지화 한국어·영어·일본어 UI와 코치 메시지
---	---

앱 크기 **약 5.2 MB** Release 기준 · 외부 3D 모델/텍스처/오디오 파일 없이 지오메트리와 효과음을 코드로 생성

느낌의 문제를 구조와 수치의 문제로 바꾸기

플레이 피드백을 재현 가능한 조건으로 바꾸고, 가장 가까운 원인을 수정했습니다.

CASE 01

최단 사거리 공격이 맞지 않음

체육관 이동 최소 반경 58이 인파이팅 보정이 적용된 왼쪽 어퍼컷 최대 적중 거리 56.6보다 컸습니다.

해결 전투 수치를 왜곡하지 않고 연습 공간의 최소 반경을 54로 조정해 모든 스타일·공격 조합의 적중을 보장했습니다.

CASE 02

샌드백이 펀치 쪽으로 흔들림

회전축 부호를 직감으로 정한 결과 펀치 반대가 아닌 공격자 방향으로 기울었습니다.

해결 Rodrigues 회전식으로 매달린 점의 이동 방향을 계산해 축을 $(z, -x)$ 에서 $(-z, x)$ 로 교정했습니다.

CASE 03

팔이 몸통에 묻혀 보임

스타일별 몸통 스케일이 달라졌지만 관절 좌표는 동일해, 슬러거의 어깨와 전완이 몸통 표면 안에 들어갔습니다.

해결 타원 경계식 $(x/r_x)^2 + (z/r_z)^2 > 1$ 로 관절이 몸 밖에 있는지 검증하고, 벡터 내적으로 팔꿈치 각도를 수치화했습니다.

CASE 04

AI 메시지가 판정과 반대로 안내

더킹·헛스윙 해설이 플레이어와 RIVAL의 역할을 구분하지 않아 카운터 권한을 반대로 전달했습니다.

해결 공격자·방어자 주체를 해설 함수에 전달해 “기회”와 “위험” 메시지를 분리하고 실제 판정과 일치시켰습니다.

반복 개선 방식

관찰·실기기 피드백

→

가설코드 경로 특정

→

검증수치·테스트 재현

→

수정빌드·실기기 확인

배운 점 “모션이 이상하다”는 피드백도 카메라 투영, 관절 경계, 회전축, 입력 상태처럼 검증 가능한 하위 문제로 나누면 안정적으로 개선할 수 있습니다.

프로토타입에서 App Store 정식 출시까지

각 단계의 한계를 다음 기술 선택의 근거로 사용했습니다.

단계	구현	검증한 질문
01	C++20 TUI	전투 루프, 저장, 플레이 패턴 기록이 게임으로 성립하는가
02	Web / Tkinter	8종 공격과 방향 방어가 화면과 입력으로 읽히는가
03	Python Top-Down	360도 이동, 의사 3D 카메라, 적응형 AI가 플레이 감각을 만드는가
04	Swift iOS	터치·3D·햅틱·접근성·배포까지 네이티브 제품으로 완성할 수 있는가
05	App Store	심사·가격 설정·스토어 메타데이터를 거쳐 실제 사용자에게 배포할 수 있는가

검증 결과

30

iOS 단위 회귀 테스트

1

가로형 UI 시나리오

57

Python 회귀 테스트

테스트 범위

- 8종 공격 수치와 조합 입력
- 방향 더킹과 카운터 보너스
- 가드·스테미나·탈진·백스텝
- 적응도와 습관별 행동 가중치
- 라운드 승패·학습 롤백·초기화
- 카메라 회전 안정성과 복서 구조

출시·운영

- Generic iOS / 실기기 빌드 검증
- XcodeGen 기반 재현 가능한 프로젝트
- TestFlight 외부 테스트와 심사 대응
- App Store 유료 앱 정식 출시
- 개인정보 비수집·오프라인 우선
- Game Center 리더보드 연동

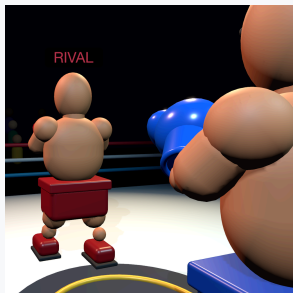
성과와 운영 판단

외부 유료 에셋과 서버를 사용하지 않아 배포 복잡도와 유지 비용을 낮췄습니다. SceneKit 노드는 최초 생성 후 재사용하고 매 프레임 자세와 카메라만 갱신합니다. 앱은 네트워크 없이 핵심 게임 전체를 실행하며, Game Center만 선택적 온라인 기능으로 분리했습니다.

테스트 수는 저장소 현재 소스 기준입니다. RIVAL 1.0은 2026년 8월 27일 한국 App Store에 정식 출시됐습니다.

직접 확인하기

코드, 플레이 영상, 실행 가능한 웹 빌드와 상세 기술 문서를 함께 제공합니다.



RIVAL

적응형 AI와 맞서는 iPhone 3D 복싱 게임

App Store 1.0 · Current source 1.1 (Build 3) · iOS 17+

항목	링크
App Store	App Store에서 RIVAL 보기
GitHub	github.com/acertainromance401/GAME
플레이 영상	YouTube에서 RIVAL 플레이 보기
웹 버전	GitHub Pages에서 바로 실행
TestFlight	RIVAL 베타 테스트
게임 설명서	ios-pixel-boxing/GAME_GUIDE.md
AI 기술 문서	ios-pixel-boxing/AI_USAGE.md

프로젝트 요약

RIVAL은 게임 규칙을 작은 프로토타입으로 검증하고, 플레이 피드백을 수치와 테스트로 전환하며, 네이티브 3D·접근성·현지화를 거쳐 App Store 정식 출시까지 완수한 개인 프로젝트입니다.